

Università degli Studi di Perugia

Dipartimento di Informatica / Ingegneria Informatica

Corso di Intelligenza Artificiale

**Analisi e Digitalizzazione di Mappe
Geopolitiche Tramite Computer Vision
e Deep Learning**

Studenti:

**Romeo Miscio 364672
Tommaso Moschini
Federico Tabuani**

Docente:

Prof.ssa Valentina Poggiani

Anno Accademico 2025/2026

Ultima modifica: 18 maggio 2026

Indice

1	Definizione del Dominio: Navigazione e Mappatura Territoriale	2
1.1	Architettura della Mappa e Discretizzazione	2
1.2	Logica di Movimento e Modello dei Costi	2
1.3	Classificazione e Tipologia dei Territori	3
2	Architettura del Software e Scelte Progettuali	3
3	Sviluppo del Modello ML: Riconoscimento dei Caratteri	4
3.1	Creazione del Modello: Dalla MLP alla CNN	4
3.2	Il Dataset EMNIST	5
3.2.1	Significato di IDX	6
3.2.2	Formato delle immagini	6
3.2.3	Formato delle label	8
3.2.4	Relazione tra immagini e label	10
3.2.5	File di Mapping	10
3.3	Tentativi di Generalizzazione e Modifiche in Inferenza	11
3.4	Filtri Anti-rumore e l'Integrazione finale di SciPy	12
4	Il Modulo Digitalizzatore: Computer Vision e Pre-elaborazione	12
4.1	Evoluzione del Programma: Dalla Base alle Soglie Adattive	12
4.2	Gestione del Tratto di Penna e Rimozione Quadretti	12
5	L'Hub Master: Integrazione e Logica di Sistema	13
5.1	Classificazione tramite Analisi delle Aree	13
5.2	Topologia degli Stati e Colorazione	14

1 Definizione del Dominio: Navigazione e Mappatura Territoriale

Il presente progetto si pone l'obiettivo di modellare un sistema di navigazione per un agente operante in un ambiente discretizzato, caratterizzato da territori frazionati e costi di transito variabili.

1.1 Architettura della Mappa e Discretizzazione

L'ambiente è rappresentato come una matrice di celle, dove la granularità della griglia determina la precisione della rappresentazione geografica:

- **Rappresentazione degli Stati:** Uno stato (o regione) può coincidere con una singola cella o essere costituito da un cluster di più celle contigue per rappresentare estensioni territoriali maggiori.
- **Segmentazione Cromatica:** Al fine di distinguere topologicamente le diverse regioni, ogni stato è caratterizzato da un colore differente rispetto a quelli adiacenti. Questo garantisce una separazione netta dei confini durante la fase di analisi dell'immagine.
- **Processo di Pixellazione:** La definizione dei confini avviene attraverso una procedura di pixellazione che mappa l'input visivo sulla griglia di navigazione (inizialmente pensata come una griglia 100×100 , in seguito resa variabile pur mantenendo le dimensioni di un quadrato), definendo l'appartenenza di ogni coordinata (x, y) a uno specifico stato.

1.2 Logica di Movimento e Modello dei Costi

Il movimento dell'agente è vincolato a spostamenti tra celle adiacenti lungo le direzioni cardinali (Nord, Sud, Est, Ovest). La funzione di costo associata alle azioni di navigazione è così definita:

- **Costo Base di Spostamento:** Il transito tra due celle adiacenti ha un costo base unitario pari a 1.
- **Penalità di Confine:** L'attraversamento di un confine tra due stati distinti comporta un onere aggiuntivo di +1 sul costo totale dello spostamento (risultando quindi in un costo totale pari a 2).
- **Penalità Territorio Nemico:** L'ingresso in uno stato ostile applica un malus aggiuntivo istantaneo al costo del tragitto. Tale malus è calcolato moltiplicando il livello di pericolosità del territorio nemico (con valori compresi tra 1 e 9) per un parametro di calibrazione X (da definire in fase di configurazione). Questo approccio parametrico permette all'algoritmo di ricerca di valutare percorsi più o meno prudenti, bilanciando in modo dinamico la brevità del tragitto con l'esposizione al rischio.

Formalizzazione Matematica ed Esempio

Dato un percorso che attraversa n celle, interseca k confini di stato e prevede l'ingresso in m stati nemici (ciascuno caratterizzato da un livello di pericolosità L_i), il costo totale C è espresso dalla seguente formula:

$$C = n + k + \sum_{i=1}^m (L_i \cdot X) \quad (1)$$

Esempio pratico: Uno spostamento attraverso 6 celle all'interno dello stesso stato alleato ha un costo pari a 6. Se lo stesso spostamento prevede l'attraversamento di un confine per entrare in uno stato nemico con livello di pericolosità 4, il calcolo del costo diverrà: 6 (costo celle)+1 (costo confine)+4X (malus nemico). Il costo totale sarà quindi pari a $7 + 4X$.

1.3 Classificazione e Tipologia dei Territori

In fase di acquisizione, ogni cella viene analizzata e classificata per definirne le proprietà fisiche e tattiche. Invece di utilizzare nomenclature estese, il sistema adotta una codifica alfanumerica ottimizzata:

Identificatore	Tipologia	Proprietà
S	Start	Punto di origine dell'agente.
W	Win	Obiettivo finale o destinazione del percorso.
A	Alleato	Territorio amichevole con transito standard.
X	Confine Naturale	Ostacolo insormontabile (non attraversabile).
1 - 9	Territorio Nemico	Rappresenta il grado di pericolosità crescente. I valori numerici sono integrati nella funzione di costo globale.

Tabella 1: Codifica alfanumerica e tipologia dei territori acquisiti

2 Architettura del Software e Scelte Progettuali

L'architettura del sistema è stata sviluppata seguendo un approccio modulare, ispirato ai principi dei microservizi, al fine di garantire scalabilità e facilità di debugging. Inizialmente concepito come uno script monolitico, il progetto è stato rifattorizzato in tre moduli distinti per evitare conflitti tra le librerie di elaborazione delle immagini e quelle di apprendimento automatico. Questa separazione delle responsabilità permette di isolare eventuali errori logici o di classificazione, garantendo che interventi sulla rete neurale non compromettano la gestione geometrica dei confini. La struttura finale si compone di:

- **Modulo Digitalizzatore** (`modulo_digitalizzatore.py`): Specializzato nelle operazioni di pre-elaborazione e computer vision. Prepara l'immagine, ispessisce l'inchiostro e rimuove i quadretti.
- **Modulo ML** (`modulo_ml.py`): Deputato al riconoscimento dei caratteri tramite reti neurali. Prende il ritaglio pulito, lo scala e restituisce il carattere predetto.

- **Hub Centrale (hub_master.py):** Agisce come controller principale del flusso di esecuzione, coordinando i moduli, isolando le isole di pixel, colorando gli stati e assemblando la matrice finale.

3 Sviluppo del Modello ML: Riconoscimento dei Caratteri

Il modulo `modulo_ml.py` funge da wrapper per una Rete Neurale Convolutionale (CNN). Per ottimizzare le prestazioni, il modello viene caricato in memoria esclusivamente all'avvio del programma.

3.1 Creazione del Modello: Dalla MLP alla CNN

Inizialmente, il Machine Learning per riconoscere lettere e numeri scritti a mano era stato strutturato con un'architettura MLP (Multilayer Perceptron). Tuttavia, i risultati non erano soddisfacenti. La criticità risiedeva nel modo in cui l'MLP elabora i dati: richiede un vettore piatto (l'appiattimento di una foto 28×28 in un vettore da 784 elementi), causando la perdita totale della percezione spaziale e geometrica delle lettere. Si è quindi deciso di passare a una CNN (Rete Neurale Convolutionale), dove le immagini vengono analizzate come griglie 2D. Il passaggio ha richiesto due modifiche: applicare la normalizzazione tra 0 e 1 e ridare all'immagine la dimensione 2D.

```
# Normalizzazione tra 0 e 1
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape dei dati per renderli compatibili con i
# layer Conv2D
X_train = X_train.reshape(-1, IMG_SIZE, IMG_SIZE,
                           1)
X_test = X_test.reshape(-1, IMG_SIZE, IMG_SIZE, 1)
```

La sostituzione del blocco MLP con l'architettura convoluzionale è stata strutturata come segue:

```
# Nuovo Modello CNN
model = keras.Sequential()
# Layer di Input: riceve immagini 28x28 con 1
# canale
model.add(layers.Input(shape=(IMG_SIZE, IMG_SIZE,
                              1)))
# Primo blocco Convoluzionale (estrae feature come
# bordi e linee)
model.add(layers.Conv2D(32, kernel_size=(3,3),
                        activation="relu"))
model.add(layers.MaxPooling2D(pool_size=(2,2)))
# Secondo blocco Convoluzionale (estrae feature
# complesse come curve e angoli)
model.add(layers.Conv2D(64, kernel_size=(3,3),
                        activation="relu"))
```

```
model.add(layers.MaxPooling2D(pool_size=(2,2)))
# Appiattisce i dati prima della classificazione
# finale
model.add(layers.Flatten())
# Dropout per evitare l'overfitting
model.add(layers.Dropout(0.5))
```

3.2 Il Dataset EMNIST

EMNIST (Extended Modified National Institute of Standards and Technology) è un celebre dataset di immagini utilizzato nel campo del machine learning e della computer vision per addestrare algoritmi a riconoscere la scrittura manuale.

Include centinaia di migliaia di **lettere dell'alfabeto** scritte a mano, oltre a tutte le immagini di **numeri**, sempre scritti a mano, contenuti nel dataset **MNIST**.

Esistono diverse versioni del dataset, dette **split** (ad esempio *Balanced*, *Letters* o *By-Class*), ciascuna contraddistinta da differenti combinazioni e bilanciamenti di lettere e numeri.

La versione che andremo a usare è la **Balanced**, in quanto offre una distribuzione equilibrata tra le varie classi di caratteri, dove ogni carattere è rappresentato da un numero simile di esempi, evitando che il modello venga addestrato molto di più su alcune classi rispetto ad altre.

Questo skip contiene **cinque** file principali:

- File con le immagini di training;
- File contenente le label associate alle immagini di training;
- File contenente le immagini di test;
- File contenente le label associate alle immagini di test;
- File di mapping.

I file delle immagini contengono i pixel dei caratteri scritti a mano, mentre i file delle label contengono il numero della classe associata a ogni immagine.

La corrispondenza tra immagini e label avviene tramite la posizione: la prima immagine ha come label il primo valore del file delle label, la seconda immagine ha come label il secondo valore, e così via.

Il file di mapping serve invece a tradurre le label numeriche interne di EMNIST nei caratteri reali.

In pratica funziona come un dizionario: associa a ogni numero di classe il codice ASCII del carattere corrispondente. Per esempio, se nel file delle label la classe 10 rappresenta la lettera A, allora nel file di mapping troveremo un'associazione tra 10 e il codice ASCII della lettera A, cioè 65. Tramite questa informazione possiamo capire che una certa immagine, associata alla label 10, rappresenta effettivamente il carattere "A".

3.2.1 Significato di IDX

La struttura generale di un file **IDX** si articola nel seguente modo:

- **Magic number** (4 byte)
- **Dimensione 1** (4 byte)
- **Dimensione 2** (4 byte)
- ...
- **Dati** (array di elementi)

Il *magic number* è un identificativo posizionato all'inizio del file che specifica la tipologia di dati contenuti. Nei file IDX occupa esattamente i primi 4 byte, la cui anatomia interna è così ripartita:

- **Primi due byte:** Sono sempre impostati a zero (00 00).
- **Terzo byte:** Indica il tipo di dato memorizzato (ad esempio, il codice 08 corrisponde al tipo *unsigned byte*).
- **Quarto byte:** Indica il numero di dimensioni del vettore o della matrice.

Per comprendere meglio, si consideri il codice esadecimale 00 00 08 03. La sua scomposizione rivela:

- 00 00 → I primi due byte fissi a zero.
- 08 → Tipo di dato *unsigned byte* (valori interi compresi tra 0 e 255, ideali per i pixel in scala di grigi).
- 03 → Array a tre dimensioni (numero di immagini, righe, colonne).

Se interpretato come un unico numero intero a 32 bit, questo valore esadecimale corrisponde al numero decimale 2051, che rappresenta il *magic number* standard per i file delle immagini nei dataset MNIST ed EMNIST. Al contrario, i file contenenti le etichette (*labels*) presentano il codice decimale 2049 (00 00 08 01): il formato dei dati rimane lo stesso (08), ma l'array è a una sola dimensione (01), in quanto si tratta di un semplice vettore lineare di valori.

3.2.2 Formato delle immagini

Nel dataset EMNIST le **immagini** sono **raffigurate** in **scala di grigi** da **28x28 pixel**.

Le immagini non sono salvate con estensioni tradizionali, come **.png** o **.jpg**, bensì in formato **IDX**. Si tratta di un formato binario semplice, progettato specificamente per rappresentare vettori e matrici multidimensionali di numeri. La struttura di un file IDX prevede una parte iniziale, denominata **header**, contenente i metadati del file, seguita dai dati veri e propri (i pixel delle immagini o i valori delle etichette).

I file di nostro interesse risultano:

- **emnist-balanced-train-images-idx3-ubyte.gz**

- **emnist-balanced-train-labels-idx1-ubyte.gz**
- **emnist-balanced-test-images-idx3-ubyte.gz**
- **emnist-balanced-test-labels-idx1-ubyte.gz**
- **emnist-balanced-mapping.txt**

Per comprendere l'organizzazione pratica di questi file, si consideri come esempio il file di addestramento:

emnist-balanced-train-images-idx3-ubyte.gz

La nomenclatura del file non è casuale, ma ne descrive dettagliatamente il contenuto e la struttura secondo lo schema seguente:

- **emnist-balanced**: Identifica lo specifico *split* (*Balanced*) del dataset EMNIST;
- **train**: Indica che i dati sono riservati alla fase di addestramento (*training set*);
- **images**: Specifica che il file contiene i pattern visivi (i pixel) e non le etichette;
- **idx3**: Indica un formato IDX a tre dimensioni;
- **ubyte**: Specifica che i dati sono memorizzati come *unsigned byte* (valori interi compresi tra 0 e 255);
- **gz**: Rappresenta l'estensione dell'archivio compresso tramite l'algoritmo *Gzip*.

L'indicazione più rilevante per il parsing dei dati è il suffisso **idx3-ubyte**. La dicitura **idx3** esplicita che i dati sono strutturati come un array tridimensionale. Nel contesto delle immagini, queste tre dimensioni corrispondono rispettivamente a:

1. Numero totale di immagini nel file;
2. Numero di righe di ciascuna immagine;
3. Numero di colonne di ciascuna immagine.

Nel caso specifico del file *EMNIST Balanced train*, la struttura del tensore dei dati può essere espressa matematicamente come una matrice tridimensionale di dimensioni $[112\,800, 28, 28]$. Ciò significa che il file racchiude:

- 112 800 immagini complessive;
- 28 righe per singola immagine;
- 28 colonne per singola immagine.

Di conseguenza, ogni singola immagine è composta da $28 \times 28 = 784$ pixel. Ciascun pixel è rappresentato da un valore numerico discreto nell'intervallo $[0, 255]$, dove lo 0 corrisponde al nero totale, il 255 al bianco puro e i valori intermedi definiscono le sfumature della scala di grigi.

Dal punto di vista fisico, all'interno del file binario non esistono elementi di separazione sintattica come indici, virgole, parentesi o metadati tra una struttura e l'altra. Il file si

presenta come un flusso continuo di dati diviso rigorosamente in due sezioni: l'header e il payload (i dati effettivi).

$$[\text{Header}] \underbrace{[\text{Pixel}, \text{Pixel}, \text{Pixel}, \text{Pixel}, \dots]}_{\text{Dati delle immagini}}$$

Nel caso specifico del file delle immagini, l'header occupa esattamente i primi 16 byte, poiché è composto da quattro numeri interi a 32 bit (ciascuno dei quali richiede 4 byte):

- **Magic number:** 4 byte (identificativo del formato).
- **Numero di immagini:** 4 byte (quantità totale di campioni).
- **Numero di righe:** 4 byte (risoluzione verticale).
- **Numero di colonne:** 4 byte (risoluzione orizzontale).

Superata la soglia dei primi 16 byte dedicati all'header, iniziano immediatamente i valori dei pixel. Il file non contiene marcatori espliciti per delimitare i confini tra le singole immagini; si sviluppa invece come un'unica sequenza unidimensionale di byte:

$$p_0, p_1, p_2, p_3, p_4, \dots, p_{88\,435\,199}$$

La separazione e la corretta attribuzione dei pixel a ciascuna immagine avvengono per via logica in fase di parsing, sapendo a priori che ogni griglia è composta da $28 \times 28 = 784$ valori consecutivi. La segmentazione del flusso binario segue quindi questa logica sequenziale:

- **Pixel da 0 a 783:** Immagine 1
- **Pixel da 784 a 1567:** Immagine 2
- **Pixel da 1568 a 2351:** Immagine 3
- ...

Questo principio evidenzia la caratteristica fondamentale del formato: all'interno del file le immagini non sono memorizzate come matrici bidimensionali, bensì come un unico vettore linearizzato (flattened) di byte compresi nell'intervallo $[0, 255]$. La geometria bidimensionale originaria (28×28) viene ricostruita via software solo in un secondo momento, sfruttando le informazioni sulle dimensioni precedentemente estratte dall'header.

3.2.3 Formato delle label

In modo analogo a quanto visto per le immagini, il file contenente le etichette di addestramento è denominato:

`emnist-balanced-train-labels-idx1-ubyte.gz`

La nomenclatura segue la medesima logica strutturale, descrivendo le proprietà intrinseche del set di dati:

- **emnist-balanced:** Identifica lo specifico *split* (*Balanced*) del dataset EMNIST.
- **train:** Indica che le etichette appartengono al set di addestramento (*training set*).

- **labels:** Specifica che il file contiene le classi corrette (i target) associate alle immagini.
- **idx1:** Indica un formato IDX a una sola dimensione.
- **ubyte:** Specifica la codifica dei dati come *unsigned byte* (valori interi compresi tra 0 e 255).
- **gz:** Rappresenta l'estensione dell'archivio compresso tramite *Gzip*.

Il suffisso chiave in questo contesto è **idx1-ubyte**. La dicitura **idx1** esplicita che i dati non sono organizzati in matrici bidimensionali, bensì strutturati come un array monodimensionale. Il file si configura quindi come un semplice vettore lineare:

$$[L_0, L_1, L_2, L_3, \dots, L_{n-1}]$$

Anche la struttura fisica di questo file prevede un header iniziale. Nel caso delle etichette, l'header occupa soltanto 8 byte, poiché deve veicolare due soli numeri interi a 32 bit (da 4 byte ciascuno):

- **Magic number:** 4 byte (identificativo del formato).
- **Numero di etichette:** 4 byte (quantità totale di campioni).

Subito dopo questa sezione di 8 byte ha inizio il flusso dei dati effettivi. La disposizione fisica del file binario si articola quindi secondo lo schema seguente:

$$[\text{Magic number}] [\text{Numero label}] \underbrace{[\text{Label}, \text{Label}, \text{Label}, \text{Label}, \dots]}_{\text{Dati delle etichette}}$$

Nel caso specifico del file *EMNIST Balanced training*, i metadati contenuti nell'header assumono i seguenti valori:

- **Magic number:** 2049 (00 00 08 01 in esadecimale, che denota il tipo *unsigned byte* mappato su un array monodimensionale).
- **Numero di etichette:** 112,800 (coerente con il numero totale di immagini presenti nel rispettivo file di payload).

Al superamento dei primi 8 byte dell'header, il file presenta una sequenza continua di 112800 singoli byte. Ciascun byte rappresenta la classe di appartenenza (il valore target) della rispettiva immagine, creando una corrispondenza biunivoca perfetta basata esclusivamente sull'indice di posizione: l'etichetta in posizione i -esima nel file delle label identificherà la classe dell'immagine ricostruita in posizione i -esima nel file delle immagini.

È importante sottolineare che un'etichetta (*label*) non è memorizzata direttamente sotto forma di carattere testuale (come "A" o "C"). All'interno del file binario, i target sono rappresentati esclusivamente da numeri interi, i quali fungono da indici interni per le classi del dataset EMNIST. Ad esempio, un'etichetta può assumere valori discreti come:

$$\text{label} = 10 \quad \text{oppure} \quad \text{label} = 35$$

Preso singolarmente, questo valore numerico non esplicita in modo immediato il carattere grafico corrispondente. Per associare univocamente l'indice numerico alla rispettiva entità alfanumerica (sapere, ad esempio, se la classe estratta rappresenti una lettera maiuscola, minuscola o una cifra), è necessario consultare un file di dizionario aggiuntivo, denominato file di mapping (*mapping file*).

3.2.4 Relazione tra immagini e label

I file delle immagini e delle etichette non contengono puntatori interni o ID per legarsi tra loro; il loro accoppiamento avviene esclusivamente per corrispondenza posizionale (o per indice). Ciò implica che la struttura logica del dataset si basa sul perfetto allineamento sequenziale dei due flussi di dati:

Immagine 0 $\longrightarrow$ Label 0
Immagine 1 $\longrightarrow$ Label 1
Immagine 2 $\longrightarrow$ Label 2
:
:

Di conseguenza, la prima immagine estratta dal file delle immagini ha come risposta corretta (*ground truth*) la prima etichetta estratta dal file delle etichette. Mentre a livello hardware (nei file grezzi) i dati si presentano come un vettore continuo di pixel e un vettore continuo di singoli byte — in cui a ogni blocco di 784 pixel corrisponde una singola etichetta —, in fase di sviluppo software questa struttura viene rimodellata. Attraverso il codice di preprocessing, i dati vengono organizzati in due array paralleli, convenzionalmente denominati X_{train} (le caratteristiche o *features*) e y_{train} (i target o *labels*), stabilendo una corrispondenza biunivoca uno-a-uno:

- $X_{\text{train}}[0]$ $\rightarrow$ Matrice 28×28 della prima immagine.
- $y_{\text{train}}[0]$ $\rightarrow$ Classe numerica corretta della prima immagine.
- $X_{\text{train}}[1]$ $\rightarrow$ Matrice 28×28 della seconda immagine.
- $y_{\text{train}}[1]$ $\rightarrow$ Classe numerica corretta della seconda immagine.
- $X_{\text{train}}[2]$ $\rightarrow$ Matrice 28×28 della terza immagine.
- $y_{\text{train}}[2]$ $\rightarrow$ Classe numerica corretta della terza immagine.

Il mantenimento di questa rigorosa corrispondenza per indice è una condizione fondamentale e imprescindibile per la fase di addestramento: se l'ordine dei vettori venisse alterato anche solo di una posizione, il modello riceverebbe associazioni errate (*mismatch*), compromettendo del tutto la capacità dell'algoritmo di generalizzare e apprendere correttamente la relazione tra i pattern visivi e le rispettive classi.

3.2.5 File di Mapping

A differenza dei file precedentemente analizzati, il file di configurazione:

`emnist-balanced-mapping.txt`

non adotta il formato binario IDX, bensì si presenta come un comune file di testo (*plain text*). La sua funzione è puramente relazionale: funge da tabella di raccordo per convertire le etichette numeriche del dataset nei rispettivi caratteri alfanumerici reali. L'organizzazione interna del file prevede che ogni riga sia composta da una coppia di valori numerici separati da uno spazio:

[indice_classe] [codice_ASCII]

Il primo valore rappresenta l'indice della classe associato internamente da EMNIST, mentre il secondo valore esprime il corrispettivo punto di codifica nello standard ASCII. Ad esempio, l'estrazione di una riga come la seguente:

10 65

indica che alla classe EMNIST 10 corrisponde il codice ASCII 65. Nella codifica dei caratteri, il valore decimale 65 mappa univocamente la lettera maiuscola 'A'. Seguendo la medesima logica, il file definisce le corrispondenze per l'intero set di classi dello split, come illustrato nei seguenti esempi:

- ...
- 12 67 $\longrightarrow$ 'C'
- 32 87 $\longrightarrow$ 'W'
- 1 49 $\longrightarrow$ '1'
- 2 50 $\longrightarrow$ '2'
- ...

In sintesi, il file di mapping descrive una pipeline di trasformazione deterministica che consente di tradurre gli output numerici del modello predittivo in caratteri testuali leggibili dall'utente umano. La sequenza logica di decodifica può essere così schematizzata:

$$\text{Label numerica EMNIST} \xrightarrow{\text{Mapping}} \text{Codice ASCII} \xrightarrow{\text{Decoding}} \text{Carattere reale}$$

$$10 \longrightarrow 65 \longrightarrow \text{'A'}$$

3.3 Tentativi di Generalizzazione e Modifiche in Inferenza

Dopo la prima correzione, la precisione era di 3 previsioni corrette e 4 errate su esempi manuali. Per migliorare la generalizzazione sono stati tentati diversi approcci:

- **Data Augmentation:** Si è provato ad aggiungere distorsioni casuali ma leggere (RandomRotation fino al 5%, RandomZoom e RandomTranslation fino al 10%) nel training set. La modifica non ha portato alcun miglioramento allo score ed è stata annullata.
- **Filtro di Sfocatura (Gaussian Blur):** In fase di inferenza si è tentato di "ammorbidire" il tratto per simularne uno simile all'inchiostro del dataset, utilizzando `img.filter(ImageFilter.GaussianBlur(radius=0.8))`. Anche questo tentativo è stato scartato per scarsi risultati.
- **Standardizzazione dell'Input:** La scelta vincente (che ha portato il punteggio a 6 caratteri corretti su 7) è stata processare le immagini di test in inferenza esattamente come nel training set: ritaglio delle parti inutili, centratura della lettera e scalatura mantenendo il rapporto d'aspetto, con lato massimo fissato a 20 pixel all'interno del riquadro 28×28 . Grazie alla precisione geometrica dell'Hub, si è potuto evitare l'uso di librerie pesanti come SciPy per la sola ricerca delle forme.

3.4 Filtri Anti-rumore e l'Integrazione finale di SciPy

Rimaneva un problema di robustezza al rumore: un semplice puntino isolato vicino a un "9" ingannava il sistema, facendogli credere che il carattere iniziasse molto più in alto. Di conseguenza il "9" veniva schiacciato in basso e predetto erroneamente come un "6" sbilenco. Inizialmente si è alzata la soglia di rilevamento dell'inchiostro, ignorando i grigi scuri e richiedendo almeno 2 pixel vicini:

```
# Ricerca intelligente: soglia aumentata a 50
righe_con_pixel = np.sum(arr > 50, axis=1) > 2
colonne_con_pixel = np.sum(arr > 50, axis=0) > 2
```

Poiché l'errore persisteva in alcuni casi limite, si è infine deciso di adottare un approccio più drastico utilizzando la libreria **SciPy** (già disponibile tramite scikit-learn) per rilevare ed isolare esclusivamente l'isola di pixel più grande associata al carattere, eliminando completamente i puntini di rumore. Questa modifica ha portato il ML a una precisione del 100%.

4 Il Modulo Digitalizzatore: Computer Vision e Pre-elaborazione

Data la natura della mappa (disegnata a mano), si è optato per tecniche di pulizia dell'immagine tramite la libreria OpenCV al posto di addestrare un ML specifico per la digitalizzazione geografica. Il modulo ha il compito di trasformare una fotografia grezza in una matrice binaria pulita.

4.1 Evoluzione del Programma: Dalla Base alle Soglie Adattive

L'idea iniziale prevedeva la conversione in scala di grigi, il rimpicciolimento su griglia 100×100 e una soglia fissa (Threshold). Tuttavia, su fogli a quadretti e senza flash i risultati erano inutilizzabili. Si è passati quindi a una **Soglia Adattiva** (`cv2.adaptiveThreshold`), calcolando il contrasto su finestre locali di 20×20 (poi affinate a 31×31 pixel). Questo rende il programma intelligente nel distinguere sfondi ombreggiati dai tratti reali. A ciò è stato affiancato un **Median Blur** (intensità 11) per cancellare le linee sottili dei quadretti mantenendo intatti i tratti marcati.

4.2 Gestione del Tratto di Penna e Rimozione Quadretti

Un ostacolo critico era l'uso di una penna dal tratto molto sottile, simile allo spessore della griglia del foglio, accentuato dalla riduzione di risoluzione.

1. **Tentativo di Dilatazione:** Si è tentato di usare un pennello 3×3 (`cv2.dilate`) per spalmare l'inchiostro e unire i buchi. Il tratto diveniva perfetto, ma il rumore aumentava drasticamente.
2. **Erosione Morfologica:** Per ottimizzare il sistema salvando i tratti deboli senza conservare i quadretti, si è optato per l'Erosione Morfologica *prima* di sfocare l'immagine. Usando un kernel ridotto, si espandono i pixel neri (ispessendo la penna) permettendo poi di alzare il livello del Median Blur in sicurezza.

```
# Creiamo un pennello 2x2
kernel_penna = np.ones((2,2), np.uint8
)
# L'erosione in scala di grigi rende i
  tratti scuri piu spessi
img = cv2.erode(img, kernel_penna,
  iterations=1)
```

3. **Chiusura dei Buchi:** Infine, è stata applicata la Chiusura Morfologica per saldare perfettamente le discontinuità nel tratto senza ingrassare eccessivamente la linea.

```
# MORPH_CLOSE salda i buchi interni al
  tratto
kernel_chiusura = np.ones((5,5), np.
  uint8)
img_binaria = cv2.morphologyEx(
  img_binaria, cv2.MORPH_CLOSE,
  kernel_chiusura, iterations=2)
```

È importante sottolineare la rimozione della funzione di pulizia globale (precedentemente denominata "Vacuum Cleaner"). Sebbene efficace nel rimuovere artefatti esterni, essa avrebbe eliminato anche i caratteri interni agli stati, fondamentali per l'etichettatura.

5 L'Hub Master: Integrazione e Logica di Sistema

L'Hub Master coordina i moduli e risolve le criticità algoritmiche emerse durante l'unione delle varie parti. Nel primo test completo vi fu un'esplosione di falsi positivi poiché mancava la rimozione del rumore globale.

5.1 Classificazione tramite Analisi delle Aree

Per filtrare gli elementi, è stata introdotta una classificazione basata sulla funzione `connectedComponent` valutando l'area in pixel delle "isole":

- **Area < 300/400:** Classificata come rumore o "polvere" e scartata.
- **Area compresa tra 300/400 e 15.000:** Identificata come potenziale carattere alfanumerico (in grado di sopravvivere alla pulizia) e inviata al modulo ML.
- **Area > 15.000:** Identificata come confine geografico o stato.

Eventuali falsi positivi residui (come un "7" rilevato nel rumore esterno alla mappa) vengono neutralizzati dalla funzione `cv2.pointPolygonTest`, la quale garantisce che solo i caratteri contenuti *all'interno* di un poligono chiuso vengano considerati nel calcolo dei percorsi.

5.2 Topologia degli Stati e Colorazione

Un problema iniziale riguardava l'uso della funzione `cv2.RETR_EXTERNAL`, che analizzava unicamente il contorno perimetrale globale. Per separare e colorare individualmente gli stati, si è scartato l'uso di algoritmi complessi (come il Teorema dei Quattro Colori) a favore della semplice assegnazione di colori unici da una palette predefinita di 12 toni. La soluzione topologica definitiva ha previsto l'inversione dello spazio di ricerca: invece di concentrarsi sulle linee nere, l'algoritmo utilizza `connectedComponents` per identificare gli spazi bianchi vuoti isolati (usando i confini neri come ostacoli), definendoli in modo perfetto come territori distinti a cui assegnare gli identificativi cromatici.